

```

In [13]: import re
import numpy as np
import pandas as pd

raw = pd.read_csv('week01_customers.csv', dtype=str)

NULL_TOKENS = ["NaN", "unknown", "-"]

CITY_ALIASES = {
    "nyc": "New York",
    "new york city": "New York",
}

DATE_FORMATS = [
    "%Y-%m-%d",
    "%m/%d/%Y",
    "%b %d %Y",
]

# Claude Used
EMAIL_RE = re.compile(r"^[^@\\s]+@[^@\\s]+\\. [A-Za-z]{2,}$")

MEMBERSHIP_ORDER = ["bronze", "silver", "gold"]

AGE_MIN, AGE_MAX = 16, 100

AGE_WORDS = {
    "zero": 0, "one": 1, "two": 2, "three": 3, "four": 4,
    "five": 5, "six": 6, "seven": 7, "eight": 8, "nine": 9,
    "ten": 10, "eleven": 11, "twelve": 12, "thirteen": 13, "fourteen": 14,
    "fifteen": 15, "sixteen": 16, "seventeen": 17, "eighteen": 18,
    "nineteen": 19, "twenty": 20, "thirty": 30, "forty": 40, "fifty": 50,
    "sixty": 60, "seventy": 70, "eighty": 80, "ninety": 90,
}

COLUMN_ORDER = [
    "customer_id",
    "name", "first_name", "last_name",
    "age",
    "city",
    "signup_date",
    "price",
    "membership",
    "email", "email_valid",
]

ID_PATTERN = r"^[C-\\d{4}]$"

CANONICAL_CITIES = ["Atlanta", "Boston", "Chicago", "New York", "Seattle"]

SIGNUP_EARLIEST = pd.Timestamp("2020-01-01")

MAX_MISSING_RATE = 0.20

```

Part 1

Task 1

Every column is an object because they are either all strings or mixed datatypes. Age should be numeric, but some people have spelled out their age, making it a string Price should be numeric, but sometimes there is a \$ in front of price

Task 2

`df.isna().sum()` does not give the full picture, because some values are not usable, yet do not show up through the command

Task 3

C-1005 and C-1013 are repeated. They are completely the same for every value

Task 4

Naive counts shown in the last cell Real counts: Boston 12 Chicago 11 Seattle 11 Atlanta 10 New York 13

```
In [14]: print(raw.shape)
print("_")
print(raw.dtypes)
print("_")
print(raw.isna().sum())
```

```
(57, 8)
```

```
_
customer_id    object
name           object
age            object
city           object
signup_date    object
price          object
email          object
membership     object
dtype: object
```

```
_
customer_id    0
name           0
age            5
city           0
signup_date    0
price          4
email          4
membership     0
dtype: int64
```

```
In [15]: print(raw)
```

	customer_id	name	age	city	signup_date	price
\						
0	C-1001	Ann Smith	25	New York	2026-01-15	\$1,299.00
1	C-1002	Brian Ochoa	thirty	Boston	2026-01-17	45.50
2	C-1003	Carla Reyes	41	Chicago	2026-01-19	230.00
3	C-1004	Devon Lee	NaN	Boston	2026-01-21	\$89.99
4	C-1005	Elena Ford	34	new york	2026-01-22	1,050.00
5	C-1006	Frank Nguyen	28	Chicago	01/23/2026	62.75
6	C-1007	Grace Adeyemi	NaN	Seattle	2026-01-25	310.20
7	C-1008	Hector Diaz	52	Boston	2026-01-26	USD 75.00
8	C-1009	Imani Brooks	forty-two	Atlanta	2026-01-28	\$2,480.00
9	C-1010	Jonas Weber	39	Seattle	2026-01-29	18.00
10	C-1011	Keisha Palmer	31	Atlanta	2026-02-01	NaN
11	C-1012	Luis Marchetti	-3	Chicago	2026-02-02	144.10
12	C-1013	Mona Haddad	27	NYC	2026-02-03	\$540.00
13	C-1014	Noah Kim	45	Boston	2026-02-05	NaN
14	C-1015	Olivia Santos	999	Seattle	2026-02-06	76.40
15	C-1016	Peter Owusu	36	Atlanta	Feb 7 2026	1,875.25
16	C-1017	Quinn Baptiste	22	Chicago	2026-02-09	\$33.00
17	C-1018	Rosa Delgado	58	new york	2026-02-10	410.00
18	C-1019	Samuel Achebe	29.0	Boston	2026-02-11	95.60
19	C-1020	Tara Whitfield	47	Seattle	2026-02-12	\$1,120.00
20	C-1021	Umar Sesay	33	Atlanta	2026-02-14	58.85
21	C-1022	Vera Lindqvist	unknown	Chicago	2026-02-15	267.30
22	C-1023	Wesley Tran	40	NEW YORK	2026-02-16	\$720.00
23	C-1024	Xiomara Peña	26	Boston	2026-02-18	49.99
24	C-1025	Yusuf Bello	61	Seattle	2026-02-19	1,340.75
25	C-1026	Zara Mitchell	35	Atlanta	2026-02-20	\$210.00
26	C-1027	Aaron Cole	NaN	Chicago	2026-02-22	88.20
27	C-1028	Bianca Rossi	38	new york	2026-02-23	615.40
28	C-1029	Caleb Johnson	24	Boston	02/24/2026	\$27.50
29	C-1030	Dalia Farouk	50	Seattle	2026-02-25	980.00
30	C-1031	Emeka Nwosu	43	Atlanta	2026-02-26	132.65
31	C-1032	Farah Aziz	twenty	Chicago	2026-02-27	\$455.00
32	C-1033	Gabriel Moreau	37	NYC	2026-03-01	70.10
33	C-1034	Hannah Osei	30	Boston	2026-03-02	NaN
34	C-1035	Ibrahim Toure	55	Seattle	2026-03-03	1,690.00
35	C-1036	Julia Kaminski	23	Atlanta	2026-03-05	\$41.25
36	C-1037	Kwame Boateng	46	Chicago	2026-03-06	525.90
37	C-1038	Lena Petrov	NaN	new york	2026-03-07	163.00
38	C-1039	Marcus Hill	32	Boston	2026-03-09	\$2,015.00
39	C-1040	Nadia Chowdhury	48	Seattle	2026-03-10	84.45
40	C-1041	Omar Haddad	21	Atlanta	2026-03-11	-
41	C-1042	Priya Raman	44	Chicago	2026-03-12	377.80
42	C-1043	Quentin Dubois	57	NEW YORK	2026-03-13	\$66.00
43	C-1044	Rachel Amoah	29	Boston	Mar 15 2026	1,208.55
44	C-1045	Silas Berhane	42	Seattle	2026-03-16	199.99
45	C-1046	Tamara Novak	NaN	Atlanta	2026-03-17	53.30
46	C-1047	Uche Okafor	51	Chicago	2026-03-18	\$845.00
47	C-1048	Valeria Cruz	26	new york	2026-03-19	112.70
48	C-1049	Wanjiru Kamau	39	Boston	2026-03-20	690.00
49	C-1050	Xander Ellis	sixty	Seattle	2026-03-22	\$38.40
50	C-1051	Yara Mansour	34	Atlanta	2026-03-23	1,455.00
51	C-1052	Zeke Alvarez	49	Chicago	2026-03-24	247.15
52	C-1053	Amara Diallo	27	NYC	2026-03-25	NaN
53	C-1054	Ben Kowalski	53	Boston	2026-03-26	\$903.60
54	C-1055	Chidi Eze	31	Seattle	2026-03-27	74.80
55	C-1005	Elena Ford	34	new york	2026-01-22	1,050.00
56	C-1013	Mona Haddad	27	NYC	2026-02-03	\$540.00

	email	membership
0	ann.smith@example.com	gold
1	b.ochoa@example.com	silver
2	carla.r@example.com	gold
3	devon.lee@example.com	bronze
4	e.ford@example.com	gold
5	NaN	silver
6	grace.a@example.com	bronze
7	hector.d@example.com	silver
8	imani.b@example.com	gold
9	jonas.w@example.com	bronze
10	keisha.p@example.com	silver
11	luis.m@example.com	bronze
12	mona.h@example.com	gold
13	noah.kim@example.com	silver
14	olivia.s@example.com	bronze
15	peter.o@example.com	gold
16	NaN	bronze
17	rosa.d@example.com	silver
18	sam.a@example.com	bronze
19	tara.w@example.com	gold
20	umar.s@example.com	silver
21	vera.l@example.com	bronze
22	wes.tran@example.com	gold
23	xio.p@example.com	bronze
24	yusuf.b@example.com	gold
25	NaN	silver
26	aaron.c@example.com	bronze
27	bianca.r@example.com	gold
28	caleb.j@example.com	bronze
29	dalia.f@example.com	silver
30	emeka.n@example.com	bronze
31	farah.a@example.com	gold
32	gabriel.m@example.com	silver
33	hannah.o@example.com	bronze
34	ibrahim.t@example.com	gold
35	julia.k@example.com	bronze
36	kwame.b@example.com	silver
37	lena.p@example.com	bronze
38	marcus.h@example.com	gold
39	nadia.c@example.com	silver
40	omar.h@example.com	bronze
41	priya.r@example.com	gold
42	NaN	silver
43	rachel.a@example.com	gold
44	silas.b@example.com	bronze
45	tamara.n@example.com	silver
46	uche.o@example.com	gold
47	valeria.c@example.com	bronze
48	wanjiru.k@example.com	silver
49	xander.e@example.com	bronze
50	yara.m@example.com	gold
51	zeke.a@example.com	silver
52	amara.d@example.com	bronze
53	ben.k@example.com	gold
54	chidi.e@example.com	bronze
55	e.ford@example.com	gold
56	mona.h@example.com	gold

```
In [16]: print(len(raw))  
         print(raw['customer_id'].nunique())  
         print(raw['customer_id'].duplicated(keep=False))
```

```
57
55
0      False
1      False
2      False
3      False
4      True
5      False
6      False
7      False
8      False
9      False
10     False
11     False
12     True
13     False
14     False
15     False
16     False
17     False
18     False
19     False
20     False
21     False
22     False
23     False
24     False
25     False
26     False
27     False
28     False
29     False
30     False
31     False
32     False
33     False
34     False
35     False
36     False
37     False
38     False
39     False
40     False
41     False
42     False
43     False
44     False
45     False
46     False
47     False
48     False
49     False
50     False
51     False
52     False
53     False
54     False
55     True
56     True
Name: customer_id, dtype: bool
```

```
In [17]: print(raw["city"].value_counts())
```

```
Boston      11
Chicago     11
Seattle     11
Atlanta     10
new york     6
NYC          4
NEW YORK     2
New York     1
Boston       1
Name: city, dtype: int64
```

Part 2

Task 5

I included NaN, unknown, and -, because they are the tokens that represent an unknown value in this dataset

Task 6

a) None are UNUSABLE, but some are null / NaN (5) b) C-1041 c) There will be many different types of unusable numbers, you must take into account a wide range

Task 7

a) Easier to understand / consolidate b) The naive call discarded null values c) Between 16 and 100 because a person cannot physically buy things outside of this range d) The mean before my range was much larger because of the 999 outlier

Task 8

Final counts same as before, .str.title is insufficient because it does not take into account apostrophes and other particularities.

Task 9

There is YYYY-MM-DD, MM/DD/YYYY, and MM DD YYYY. 01/23/2026 is unambiguous because the beginning month can only be a month not a day, since 23 cannot be a month

Task 10

4 addresses are missing (C-1006, C-1017, C-1026, C-1043). Of the 53 present, all 53 pass the regex. Gabriel Garcia Marquez breaks because it is 3 parts Mao Zedong breaks because family name comes first

```

In [18]: def to_null(value):

    if isinstance(value, str):
        value = value.strip()
    if value in NULL_TOKENS:
        return np.nan

    return value

def clean_price(value):
    if isinstance(value, str):
        value = value.strip()
        value = value.strip("$")
        value = value.strip("USD")
        value = value.replace(",", "")
        value = float(value)

    return value

def clean_age(value):
    value = to_null(value)
    if not isinstance(value, str):
        return float(value)

    text = re.sub(r"\s+", " ", value.strip().lower())
    try:
        return float(text)
    except ValueError:
        pass

    # spelled-out ages ("thirty", "forty-two") are recoverable, not junk
    total = 0.0
    for word in re.split(r"[-\s]+", text):
        if word not in AGE_WORDS:
            return np.nan
        total += AGE_WORDS[word]

    return total

def validate_age(value):
    value = to_null(value)
    if value < AGE_MIN or value > AGE_MAX:
        value = np.nan

    return value

def clean_city(value):
    value = to_null(value)
    if not isinstance(value, str):
        return value

    value = value.strip()
    value = re.sub(r"\s+", " ", value)

    key = value.lower()
    if key in CITY_ALIASES:
        return CITY_ALIASES[key]

    return value.title()

```



```
def clean_date(value):
    value = to_null(value)
    if not isinstance(value, str):
        return pd.NaT

    value = value.strip()
    value = re.sub(r"\s+", " ", value)

    for fmt in DATE_FORMATS:
        try:
            return pd.to_datetime(value, format=fmt)
        except ValueError:
            pass

    return pd.NaT

def valid_email(value):
    value = to_null(value)
    if not isinstance(value, str):
        return False

    return EMAIL_RE.match(value.strip()) is not None
```

```
In [19]: # email validity
raw["email_valid"] = raw["email"].map(valid_email)

present = raw["email"].notna()
print("missing emails:", int((~present).sum()))
print("present emails:", int(present.sum()))
print("present and passing:", int((present & raw["email_valid"]).sum()))
print("present and failing:", int((present & ~raw["email_valid"]).sum()))

# split name into first / last
n_parts = raw["name"].str.strip().str.split(r"\s+", regex=True).str.len()
print("every row splits into exactly two parts:", bool(n_parts.eq(2).all()))
print(n_parts.value_counts())

parts = raw["name"].str.strip().str.split(r"\s+", expand=True, regex=True)
raw["first_name"] = parts[0]
raw["last_name"] = parts[1]

print(raw[["name", "first_name", "last_name", "email", "email_valid"]].head())

missing emails: 4
present emails: 53
present and passing: 53
present and failing: 0
every row splits into exactly two parts: True
2      57
Name: name, dtype: int64
```

	name	first_name	last_name	email	email_valid
0	Ann Smith	Ann	Smith	ann.smith@example.com	True
1	Brian Ochoa	Brian	Ochoa	b.ochoa@example.com	True
2	Carla Reyes	Carla	Reyes	carla.r@example.com	True
3	Devon Lee	Devon	Lee	devon.lee@example.com	True
4	Elena Ford	Elena	Ford	e.ford@example.com	True

Part 3

Task 11

`raw.copy()` matters so that we don't apply to the actual dataframe everytime we run this function. If we run the function twice, it won't really do anything on the second pass.

Task 12

Both row counts are the same because this dataset has two rows that are completely identical. They would disagree if there were two people with the same id but different information. `keep="first"` would be best in that scenario.

Task 13

An assertion that never fails is worth writing so others can have a greater understanding of your code.

```
In [20]: def clean(raw):
    res = raw.copy()

    # apply functions for each column
    res["customer_id"] = res["customer_id"].map(to_null)

    res["name"] = res["name"].map(to_null)
    parts = res["name"].str.split(r"\s+", n=1, expand=True, regex=True)
    res["first_name"] = parts[0]
    res["last_name"] = parts[1]

    res["age"] = res["age"].map(clean_age).map(validate_age)

    res["city"] = res["city"].map(clean_city)
    res["signup_date"] = res["signup_date"].map(clean_date)

    res["price"] = res["price"].map(to_null).map(clean_price)

    # ordered categorical for membership
    res["membership"] = pd.Categorical(
        res["membership"].map(to_null).str.lower(),
        categories=MEMBERSHIP_ORDER,
        ordered=True,
    )

    # flag bad emails instead of dropping the customer (Task 10)
    res["email"] = res["email"].map(to_null)
    res["email_valid"] = res["email"].map(valid_email)

    # remove duplicates
    res = res.drop_duplicates().reset_index(drop=True)

    # return with columns in proper order
    return res[COLUMN_ORDER]
```

```
In [21]: # Claude Used
def validate(df):
```

```

# --- structure ---
missing = [c for c in COLUMN_ORDER if c not in df.columns]
unexpected = [c for c in df.columns if c not in COLUMN_ORDER]
assert list(df.columns) == COLUMN_ORDER, \
    f"columns changed (missing={missing}, unexpected={unexpected})"
assert len(df) > 0, "cleaning returned an empty frame"
assert df["customer_id"].notna().all(), "a row has no customer_id"
assert df["customer_id"].is_unique, \
    f"duplicate ids survived dedup: {df.loc[df['customer_id'].duplicated()]}
well_formed = df["customer_id"].str.match(ID_PATTERN)
assert well_formed.all(), \
    f"customer_id format changed: {df.loc[~well_formed, 'customer_id'].tolist()}

# --- dtypes ---
assert pd.api.types.is_float_dtype(df["age"]), f"age is {df['age'].dtype},
assert pd.api.types.is_float_dtype(df["price"]), f"price is {df['price'].dtype},
assert pd.api.types.is_datetime64_any_dtype(df["signup_date"]), \
    f"signup_date is {df['signup_date'].dtype}, not datetime"
assert isinstance(df["membership"].dtype, pd.CategoricalDtype), \
    f"membership is {df['membership'].dtype}, not categorical"
assert df["membership"].cat.ordered, "membership is categorical but unordered"
assert list(df["membership"].cat.categories) == MEMBERSHIP_ORDER, \
    f"membership order changed: {list(df['membership'].cat.categories)}"
assert pd.api.types.is_bool_dtype(df["email_valid"]), \
    f"email_valid is {df['email_valid'].dtype}, not bool"

# --- value ranges ---
age = df["age"].dropna()
in_range = age.between(AGE_MIN, AGE_MAX)
assert in_range.all(), \
    f"age outside [{AGE_MIN}, {AGE_MAX}]: {sorted(age[~in_range].tolist())}"
price = df["price"].dropna()
assert (price > 0).all(), f"non-positive price: {sorted(price[price <= 0].tolist())}"
dates = df["signup_date"].dropna()
assert dates.min() >= SIGNUP_EARLIEST, \
    f"signup_date before {SIGNUP_EARLIEST.date()}: {dates.min().date()}"
assert dates.max() <= pd.Timestamp.today(), \
    f"signup_date in the future: {dates.max().date()}"
assert set(df["city"].dropna()) <= set(CANONICAL_CITIES), \
    f"unrecognised city: {sorted(set(df['city'].dropna()) - set(CANONICAL_CITIES))}"

# --- coverage: a column that is mostly NaN is not usable, even if typed right
for col in ("age", "price"):
    rate = df[col].isna().mean()
    assert rate <= MAX_MISSING_RATE, \
        f"{col} is {rate:.0%} missing (limit {MAX_MISSING_RATE:.0%})"

return True

```

Part 4

Task 14

It is interesting how the median of silver is less than that of bronze. Likely because there are less customers in silver.

Task 15

Atlanta has the highest mean, New York has the highest median. Median is best for this comparison, because it is resistant to outliers.

Task 16

The largest month is March with 23 signups. It should not be evidence of growth because weeks may differ (not grow). Weekly counts might be more helpful.

```
In [22]: clean_df = clean(raw)

by_tier = clean_df.groupby("membership")["price"].agg(
    customers="size",
    priced="count",
    mean="mean",
    median="median",
).round(2)

print(by_tier)
print("\npriced rows:", int(clean_df["price"].notna().sum()), "of", len(clean_df))
```

	customers	priced	mean	median
membership				
bronze	21	18	109.06	89.10
silver	16	14	255.64	79.72
gold	18	18	1123.35	1085.00

priced rows: 50 of 55

```
In [23]: by_city = clean_df.groupby("city")["price"].agg(
    customers="size",
    priced="count",
    mean="mean",
    median="median",
).round(2)

print(by_city)
print("\npriced rows:", int(clean_df["price"].notna().sum()), "of", len(clean_df))
```

	customers	priced	mean	median
city				
Atlanta	10	8	788.29	171.32
Boston	12	10	520.07	92.79
Chicago	11	11	297.84	247.15
New York	11	10	504.62	475.00
Seattle	11	11	539.36	199.99

priced rows: 50 of 55

```
In [24]: d = clean_df["signup_date"]
print("dated rows:", int(d.notna().sum()), "of", len(clean_df))
print("window:", d.min().date(), "to", d.max().date())

per_month = d.dt.to_period("M").value_counts().sort_index()
print(per_month)
print("largest month:", per_month.idxmax(), "=", per_month.max())
```

```

dated rows: 55 of 55
window: 2026-01-15 to 2026-03-27
2026-01    10
2026-02    22
2026-03    23
Freq: M, Name: signup_date, dtype: int64
largest month: 2026-03 = 23

```

```

In [25]: fresh = pd.read_csv("week01_customers.csv", dtype=str, keep_default_na=False)

print("ROWS      ", len(clean_df), "customers (", len(fresh), "raw -",
      int(fresh.duplicated().sum()), "exact duplicates)")
print("      duplicate ids:", fresh.loc[fresh.duplicated(), "customer_id"])
print("      customer_id unique:", clean_df["customer_id"].is_unique)

print("\nCITIES      ", clean_df["city"].nunique(), clean_df["city"].value_counts())

print("\nDATES      ", clean_df["signup_date"].min().date(), "to",
      clean_df["signup_date"].max().date(),
      "| unparsed:", int(clean_df["signup_date"].isna().sum()))

age = clean_df["age"]
print("\nAGE      usable", int(age.notna().sum()), "of", len(clean_df),
      "| mean", round(age.mean(), 1), "| range", age.min(), "-", age.max())
print("      blank: 6 missing in source + 2 blanked as impossible (999, -)
print("      mean before the plausibility range:",
      round(fresh.drop_duplicates()["age"].map(clean_age).mean(), 1),
      "-> after:", round(age.mean(), 1))
print("      ages recovered from words that to_numeric would drop:",
      int(fresh.drop_duplicates()["age"].map(clean_age).notna().sum())
      - pd.to_numeric(fresh.drop_duplicates()["age"], errors="coerce").notna().sum())

price = clean_df["price"]
print("\nPRICE      usable", int(price.notna().sum()), "of", len(clean_df),
      "| mean", round(price.mean(), 2), "| median", price.median())
print("      if C-1041's '-' were read as $0.00: mean",
      round(price.sum() / (price.notna().sum() + 1), 2),
      "over", int(price.notna().sum() + 1))
print("      revenue in the two removed duplicate rows:",
      round(fresh[fresh.duplicated()["price"]
      .map(to_null).map(clean_price).sum(), 2))

print("\nMEMBERSHIP ", clean_df["membership"].value_counts().to_dict(),
      "| missing:", int(clean_df["membership"].isna().sum()))
print("EMAIL      present", int(clean_df["email"].notna().sum()), "of", len(clean_df),
      "| well-formed", int(clean_df["email_valid"].sum()))

print("\nDEFECT SCOPE (raw rows affected)")
print(" price with $:", int(fresh["price"].str.contains(r"\$").sum()),
      "| with thousands separator:", int(fresh["price"].str.contains(",").sum()),
      "| with USD prefix:", int(fresh["price"].str.contains("USD").sum()))
print(" age spelled out:", fresh.loc[fresh["age"].isin(AGE_WORDS)
      | fresh["age"].str.contains("-", na=False) & ~fresh["age"].str.match(r"^\d{1,2}$"),
      fresh["age"].str.contains("-", na=False) & ~fresh["age"].str.match(r"^\d{1,2}$")])
print(" city spellings:", fresh["city"].value_counts().to_dict())
print(" date formats: ISO", int(fresh["signup_date"].str.match(r"^\d{4}-\d{2}-\d{2}$").sum()),
      "| MM/DD/YYYY", int(fresh["signup_date"].str.match(r"^\d{2}/\d{2}/\d{4}$").sum()),
      "| Mon D YYYY", int(fresh["signup_date"].str.match(r"^[A-Z][a-z]{2} \d{4}$").sum()),
      "| null tokens by column:",
      {c: {v: int((fresh[c] == v).sum()) for v in ("", "N/A", "n/a", "unknown")}})

```

```

if (fresh[c] == v).any()} for c in fresh.columns
if any((fresh[c] == v).any() for v in ("", "N/A", "n/a", "unknown", "-"))

```

ROWS 55 customers (57 raw – 2 exact duplicates)
duplicate ids: ['C-1005', 'C-1013']
customer_id unique: True

CITIES 5 {'Boston': 12, 'New York': 11, 'Chicago': 11, 'Seattle': 11, 'Atlanta': 10}

DATES 2026-01-15 to 2026-03-27 | unparsed: 0

AGE usable 47 of 55 | mean 38.1 | range 20.0 – 61.0
blank: 6 missing in source + 2 blanked as impossible (999, -3)
mean before the plausibility range: 56.9 -> after: 38.1
ages recovered from words that to_numeric would drop: 4

PRICE usable 50 of 55 | mean 515.25 | median 220.0
if C-1041's '-' were read as \$0.00: mean 505.15 over 51
revenue in the two removed duplicate rows: 1590.0

MEMBERSHIP {'bronze': 21, 'gold': 18, 'silver': 16} | missing: 0

EMAIL present 51 of 55 | well-formed 51

DEFECT SCOPE (raw rows affected)
price with \$: 17 | with thousands separator: 11 | with USD prefix: 1
age spelled out: ['thirty', 'forty-two', 'twenty', 'sixty']
city spellings: {'Boston': 11, 'Chicago': 11, 'Seattle': 11, 'Atlanta': 10, 'new york': 6, 'NYC': 4, 'NEW YORK': 2, 'New York': 1, ' Boston ': 1}
date formats: ISO 53 | MM/DD/YYYY 2 | Mon D YYYY 2
null tokens by column: {'age': {'': 2, 'N/A': 3, 'unknown': 1}, 'price': {'': 3, 'n/a': 1, '-': 1}, 'email': {'': 4}}

Part 5

Task 17

999 could be because of someone specifically trying to ruin the data (adversarial input), and -1 could be a system error or even a missing value. I would need to specify with the customer who gave this info, or even speak to the people who aggregated this data. This distinction changes the way I view the customers, and also the validity of the data source.

Task 18

1. Missing value. I would make it NaN, doing nothing to the mean.
2. A. I would take it away, because there is no transaction, also doing nothing to the mean
3. The median itself. I would keep it. This would do nothing to the mean. I would stick to missing value, since it quite literally holds no data / ambiguous data. A float() call will throw an error with this value.

Task 19

They could be data entry errors because the price is the same for them. They could be legitimate purchases if the user put in the exact same information both times. Price is the same, so I assume it is a simple data entry error. I would write in a report that there is ambiguity and it needs to be taken with a grain of salt. Maybe include the data values with the two extra rows and without if there are noticeable changes to the trend.

Task 20

Data quality report — week01_customers.csv to customers_clean.csv

Defects found and what I did about them

#	Defect	Scope	Severity	Action taken
1	Four different missing-value conventions in one file: "", N/A / n/a, unknown, -	age 6 rows, price 5, email 4	High	Normalized through <code>to_null()</code> against a named <code>NULL_TOKENS</code> constant. Three of the four are invisible to <code>isna()</code> on the raw read, so a missingness check run before conversion understates the problem.
2	Exact duplicate rows	2 rows (C-1005, C-1013), identical across all 8 raw columns	High	<code>drop_duplicates()</code> , 57 → 55. See Unresolved #2 — I am not certain these are errors.
3	<code>price</code> stored as currency text	17 rows with \$, 11 with thousands separators, 1 with a <code>USD</code> prefix	High	<code>clean_price()</code> strips symbols and separators, then casts to float. Column was unusable as a number before this.
4	<code>age</code> stored as text, including spelled-out numbers	4 spelled out (thirty, forty-two, twenty, sixty), 1 written 29.0	Medium	<code>clean_age()</code> maps number words via an <code>AGE_WORDS</code> constant. This recovers 4 real ages that <code>pd.to_numeric(errors='coerce')</code> silently discards.
5	Impossible ages	2 rows: 999 (C-1015) and -3 (C-1012)	High	<code>validate_age()</code> blanks anything outside 16–100. These two rows alone moved mean age by 18.8 years (56.9 → 38.1), so this is not cosmetic.
6	<code>city</code> spelling variants	New York split across 4 spellings (13 raw rows); one Boston padded with whitespace	High	<code>clean_city()</code> with a <code>CITY_ALIASES</code> map, then title case. Before this, New York was split into four buckets and never surfaced as a major city — grouping on the raw column gives the wrong answer to "which city is largest."
7	<code>signup_date</code> in three formats	53 ISO, 2 MM/DD/YYYY, 2 Mon D YYYY	Medium	<code>clean_date()</code> tries each format in <code>DATE_FORMATS</code> in turn. Zero parse failures.
8	Missing emails	4 rows	Low	Added a boolean <code>email_valid</code> column instead of dropping rows — a missing email is not a reason to lose a customer's price and city. All 51 present addresses are well-formed.

Headline figures

- **Rows: 55 customers** (57 - 2 exact duplicates).
- **Cities: 5** (Boston 12, New York 11, Chicago 11, Seattle 11, Atlanta 10).
- **Signup dates: 2026-01-15 to 2026-03-27.**
- **Age** Mean 38.1, range 20–61. The 8 blanks are 6 missing in the source plus 2 blanked as impossible.
- **Price** Mean 515.25, *Median* 220.00. Prefer the median, the distribution is right-skewed.
- **Membership: bronze 21, silver 16, gold 18** (stored as an ordered categorical).
- **Email: 51 of 55 present** (all 51 valid).

Unresolved

These need the data owner. I could not answer any of them from the file itself.

1. **What does – mean in C-1041's price?** I read it as missing.
2. **Are C-1005 and C-1013 duplicate entries or genuine second purchases?** Both rows match their originals across all eight columns.
3. **What does price actually measure** a single purchase, lifetime value, or a recurring rate?
4. **Is 999 a system error or bad input?** If it is an error, other columns may have it too.

AI Documentation

Prompt: Write a regular expression that checks basic email structure. Gave me the proper regex, and I accepted it. It is a relatively objective task.

Prompt: write the validate(df) function from task 13. Wrote the function, and made sure the notebook passed. Accepted because it runs without issue.

Metacognitive reflection

AI improved my workflow significantly, making hard tasks easier. Whenever I had trouble with something, it was very simple to ask Claude to find a solution. In the past, I would have been completely stuck if I did not understand a concept completely. However, now I am able to use Claude to fill in the gaps and understand more fully. I always make sure that I understand a change before I allow it. I verified that Claude was correct by running the code myself and checking against online tools like stack overflow and geeks or geeks to nail down concepts. I only accepted suggestions, I did not modify or reject any.

```
In [26]: # Export -- validate before writing, so a bad frame never reaches disk
         validate(clean_df)

         clean_df.to_csv("customers_clean.csv", index=False, date_format="%Y-%m-%d")

         check = pd.read_csv("customers_clean.csv")
```



```
print("wrote customers_clean.csv:", len(check), "rows,", len(check.columns), "\n")
print(check.head())
```

wrote customers_clean.csv: 55 rows, 11 columns

	customer_id	name	first_name	last_name	age	city	signup_date	\
0	C-1001	Ann Smith	Ann	Smith	25.0	New York	2026-01-15	
1	C-1002	Brian Ochoa	Brian	Ochoa	30.0	Boston	2026-01-17	
2	C-1003	Carla Reyes	Carla	Reyes	41.0	Chicago	2026-01-19	
3	C-1004	Devon Lee	Devon	Lee	NaN	Boston	2026-01-21	
4	C-1005	Elena Ford	Elena	Ford	34.0	New York	2026-01-22	

	price	membership	email	email_valid
0	1299.00	gold	ann.smith@example.com	True
1	45.50	silver	b.ochoa@example.com	True
2	230.00	gold	carla.r@example.com	True
3	89.99	bronze	devon.lee@example.com	True
4	1050.00	gold	e.ford@example.com	True

In []: